Semana 05 — Reproducir, mitigar y verificar con el mismo payload

Grupo 1 · rol Azul · ejemplo de trabajo

Prof. César Rodríguez | ELECTRONICA | 1 de noviembre de 2026

Curso	CYBERSEGURIDAD — Ciberseguridad Aplicada
Semana	05 de 10
Fase	2 — Ofensiva controlada
Grupo	1
Rol	Azul
Entrega	GitHub, carpeta semana_05/
Corte	Domingo 23:59 de la semana correspondiente; lo posterior no se corrige.

1 Tu rol en la práctica

Parte de su propia captura cruda y aplica el control equivalente: configuración, parche o regla de red. Repite el ataque para demostrar que falla y documenta el antes y el después.

- Leer su propia evidencia cruda y elegir el control que corta el ataque en el punto más barato.
- Aplicar el control y repetir el ataque para demostrar que ya no funciona.
- Documentar el antes y el después con los mismos comandos y la diferencia medida.

2 Qué aporta este rol al entregable

La mitigación aplicada, el comando de verificación y la comparación de resultados.

3 Ejemplo paso a paso

El rol Azul reproduce la falla y despues aplica el control que la corta, sin cambiar el payload. Si el resultado cambia porque se cambio la peticion, no se demostro nada.

3.1 Iniciar sesion y fijar el nivel de DVWA

Todo se ejecuta con la sesion del laboratorio y el nivel bajo, que es el que se quiere demostrar vulnerable. El token CSRF se extrae de la pagina, no se supone.

```
mkdir -p semana_05/evidencia
curl -s -c semana_05/evidencia/cookies.txt \
  http://127.0.0.1:8081/login.php > semana_05/evidencia/login.html
TOKEN=$(grep -o "name='user_token' value='[^']*'" \
  semana_05/evidencia/login.html | head -1 | cut -d"'" -f4)
curl -s -b semana_05/evidencia/cookies.txt -c semana_05/evidencia/
    cookies.txt \
  -d "username=admin&password=password&Login=Login&user_token=$TOKEN" \
  http://127.0.0.1:8081/login.php -o /dev/null
curl -s -b semana_05/evidencia/cookies.txt \
  -d "security=low&seclev_submit=Submit" \
  http://127.0.0.1:8081/security.php -o /dev/null
```

3.2 Capturar la evidencia de los tres hallazgos

Cada peticion se guarda con su respuesta completa. El reflejo del XSS, la fila extra de la inyeccion y el contenido del archivo leido son la prueba, y se guardan tal cual.

```
curl -s -b semana_05/evidencia/cookies.txt \
  'http://127.0.0.1:8081/vulnerabilities/xss_r/?name=%3Cscript%3Ealert
     (1)%3C/script%3E' \
  > semana_05/evidencia/xss_antes.html
curl -s -b semana_05/evidencia/cookies.txt \
  'http://127.0.0.1:8081/vulnerabilities/sqli/?id=1%27+OR+%271%27%3D
     %271&Submit=Submit' \
  > semana_05/evidencia/sqli_antes.html
curl -s -b semana_05/evidencia/cookies.txt \
  'http://127.0.0.1:8081/vulnerabilities/fi/?page=../../../../etc/
     passwd' \
  > semana_05/evidencia/lfi_antes.txt
sha256sum semana_05/evidencia/*_antes.* > semana_05/evidencia/antes.
    sha256
```

3.3 Aplicar el control que elimina la causa

El control se aplica donde esta la causa: parametrizacion y codificacion en la aplicacion, o nivel alto de DVWA cuando el codigo no se puede editar. Se elige uno y se documenta el motivo.

```
curl -s -b semana_05/evidencia/cookies.txt \
  -d "security=high&seclev_submit=Submit" \
  http://127.0.0.1:8081/security.php -o /dev/null
printf 'nivel alto aplicado; se documenta que el control es de la
    aplicacion\n' \
  > semana_05/evidencia/control.txt
```

3.4 Repetir exactamente el mismo payload

Las mismas URL, el mismo orden y la misma sesion. El resultado esperado es que el reflejo se codifique, la inyeccion no devuelva filas extra y el archivo no se lea.

```
curl -s -b semana_05/evidencia/cookies.txt \
  'http://127.0.0.1:8081/vulnerabilities/xss_r/?name=%3Cscript%3Ealert
     (1)%3C/script%3E' \
  > semana_05/evidencia/xss_despues.html
grep -c '&lt;script&gt;' semana_05/evidencia/xss_despues.html
curl -s -b semana_05/evidencia/cookies.txt \
  'http://127.0.0.1:8081/vulnerabilities/fi/?page=../../../../etc/
     passwd' \
  | wc -c > semana_05/evidencia/lfi_despues.txt
```

3.5 Hallazgo, control y verificacion con el mismo payload

Hallazgo, control y verificacion con el mismo payload

Hallazgo	Control aplicado	Resultado antes	Resultado despues	Verificacion
XSS reflejado	Codificacion de salida y nivel alto	Script reflejado sin codificar	Texto codificado	xss_despues.html
Inyeccion SQL	Consulta parametrizada	Filas no previstas	Sin filas extra	sqli_despues.html
LFI	Lista blanca de archivos	Contenido de /etc/passwd	Error controlado	lfi_despues.txt

"Neutralizado" solo se escribe si la repeticion del payload lo demuestra; si el control no se pudo aplicar, se escribe "no aplicado" con el motivo.

4 Qué guardar en el repositorio

El informe muestra, por hallazgo, la peticion, la respuesta antes, el control aplicado y la respuesta despues con el mismo payload, mas el hash de la evidencia.

```
semana_05/
|-- informe.md
|-- evidencia/
|   |-- cookies.txt
|   |-- xss_antes.html
|   |-- xss_despues.html
|   |-- sqli_antes.html
|   |-- sqli_despues.html
|   |-- lfi_antes.txt
|   |-- lfi_despues.txt
|   `-- antes.sha256
`-- README.md
```

- El payload de antes y el de despues son identicos byte a byte.
- El control se aplico en la causa, no en el enunciado del informe.
- La sesion y el nivel de seguridad estan documentados.
- La evidencia esta anonimizada antes de versionarse.

5 Cómo se corrige

Criterio	Peso
Explotación reproducible de los tres tipos de vulnerabilidad	3,0
Causa raíz correctamente identificada	2,5
Mitigación concreta y verificable por hallazgo	2,0
Evidencia clara y anonimizada	1,5
Retos de Juice Shop justificados	1,0
Total	**10,0**

- {'Las cookies de sesion son credenciales': 'no se suben al repositorio.'}
- Subir el nivel de DVWA es un control de laboratorio; no sustituye corregir el codigo.

Recuerda

El trabajo es independiente: cada grupo usa su propio repositorio y su propio laboratorio, y la comparación de resultados es en clase. Corte: Domingo 23:59 de la semana correspondiente; lo posterior no se corrige.